Theoretical Computer Science

Tianlin Pan

November 11, 2025

Contents

1 数学术语 4
1.1 字符串和语言 4
1.2 可满足性问题 5
2 定理和证明 6
2.1 零知识证明 6
3 确定性有穷自动机 7
3.1 有穷自动机的定义 7
3.2 有穷自动机识别的语言 7
3.3 有穷自动机的模拟运行 7
4 正则语言 9
4.1 计算的形式化定义 9
4.2 正则语言的定义 9
4.3 非正则语言的例子 9
4.4 正则运算 9
5 非确定性有穷自动机 11
5.1 形式化定义 11
5.2 NFA 的状态转移 11
5.3 NFA 的计算过程 11
5.4 NFA 的 “猜想” 行为 12
5.5 ε-NFA 的转换 12
5.6 NFA 和 DFA 的等价性 13
6 正则表达式 15
6.1 引子 15
6.2 正则表达式的形式化定义 15
6.3 运算的优先级 16
6.4 有穷自动机和正则表达式 16
6.5 正则表达式代数定律 19
6.6 Arden 引理 20
6.7 正则表达式的应用 21
7 泵引理 (Pumping Lemma) 22
7.1 引子 22
7.2 引理内容 22
7.3 证明思路 22
7.4 使用泵引理证明语言非正则 22
8 极小化 DFA 23
8.1 引子 23
8.2 等价状态 23
8.3 划分等价类法 23
8.4 Myhill-Nerode 定理 25
8.5 判断 DFA 的等价性 29
9 文法 (Grammar) 30

9.1 引子 30
9.2 形式化定义 30
9.3 推导 (Derivation) 31
9.4 语言, 句子和句型 31
9.5 文法的构造 32
9.6 正则文法 33
9.7 空语句 (Null Sentence) 35
9.8 上下文无关文法 (CFG) 36

1 数学术语

1.1 字符串和语言

1.1.1 字母表

任意非空有穷集合:

- $\Sigma = \{0, 1\}$
- $\Sigma = \{a, b, c, ..., x, y, z, \text{space}\}$

1.1.2 (字符) 串

字母的有穷序列 - $x = 01001$, $w = \text{madam}$

字符串的 **连接**

- $xw = 01001\text{madam}$
- $xx = x^2 = 0100101001$

串的 **长度** - $|x| = |w| = 5 \Longrightarrow |xw| = 10$

空串 $|\varepsilon| = 0$, $x^0 = \varepsilon$.

1.1.3 子串和子序列

子串 要求是连续的片段 - ada 是 madam 的子串

子序列 则不要求连续 - mdm 是 madam 的子序列

1.1.4 语言

语言 是由字符串组成的集合

几种特殊的语言:

$$\begin{aligned} \Sigma^* &= \{x \mid x \in \Sigma \ \text{ and } \ |x| \ \text{ is finite}\} \\ \Sigma^+ &= \{x \mid x \in \Sigma \ \text{ and } \ |x| \ \text{ is finite and } \ |x| > 0\} \\ \Sigma' &= \{x \mid x \in \Sigma \ \text{ and } \ |x| \ \text{ is infinite}\} \end{aligned} \tag{1}$$

因此, 任何由有限长度字符串组成的语言 A 都是 Σ^* 的一个子集

$$A \subset \Sigma^* \tag{2}$$

空语言 是一个不包含任何字符串的语言, 用符号 $\emptyset$ 表示 **空串语言** 是一个包含唯一一个字符串的语言, 而这个唯一的字符串是**空串**, 符号为 $\{\varepsilon\}$.

语言的连接:

$$AB = \{xy \mid x \in A \quad \text{and} \quad y \in B\} \tag{3}$$

有两个特殊情况是

- $\{\varepsilon\}A = A\{\varepsilon\} = A$
- $\emptyset A = A\emptyset = A$

1.1.5 标准序

排序方法

- **先短后长**: 如果两个字符串的长度不同, 那么短的字符串排在前面
- **等长逐位比较**: 如果两个字符串的长度相同, 那么就从第一个字符开始, 逐个字符地比较它们的顺序

$$\begin{aligned}\Sigma_2^* = \{&\varepsilon, a, b, c, ..., x, y, z, aa, ab, ac, ...ax, ay, az, ba, ..., \\ &bz, ..., za, ..., zz, aaa, ..., zzz, aaaa, ...\}\end{aligned} \tag{4}$$

1.2 可满足性问题

是否存在一种对变量的赋值方式 (比如把某个变量设为真或假), 能让整个公式的结果为真. 如果存在, 那么这个公式就是**可满足的**.

SAT 定义为所有可满足的布尔公式的集合:

$$\text{SAT} = \{\phi \mid \phi \ \text{ is a satisfiable Boolean expression } \} \tag{5}$$

CNF 公式 也就是 **合取范式**. 一个布尔公式如果是由若干个子句通过与 ($\wedge$) 连接起来的, 那么它就是一个 CNF 公式. 特别地, 如果一个 CNF 公式中的所有字句都有三个文字, 那么这个公式就被称为 **3-CNF 公式**. 例如:

$$(x_1 \vee \bar{x}_2 \vee \bar{x}_3) \wedge (x_3 \vee \bar{x}_5 \vee x_6) \wedge (x_3 \vee \bar{x}_6 \vee x_4) \tag{6}$$

3SAT 问题:

$$\text{3SAT} = \{\phi \mid \phi \ \text{ is a satisfiable 3-CNF } \} \tag{7}$$

2 定理和证明

2.1 零知识证明

零知识证明是一种协议, 它允许一个人向另一个人证明某个陈述是真实的, 而无需透露任何关于这个陈述的额外信息. 可以把它想象成一个谜题, **证明者**知道谜底, 他要让**验证者**相信他确实知道谜底, 但同时又不能把谜底告诉对方.

3 确定性有穷自动机

3.1 有穷自动机的定义

一个确定性有穷自动机 (Deterministic Finite Automaton, DFA) 是一个五元组 $(Q, \Sigma, \delta, q_0, F)$, 其中

1. Q 是一个有限 **状态** 集合.
2. Σ 是一个有限 **输入符号** 集合, 称为 **字母表**.
3. $\delta : Q \times \Sigma \to Q$ 是一个 **状态转移函数**, 它定义了在给定当前状态和输入符号的情况下, 自动机将转移到哪个状态.
4. $q_0 \in Q$ 是 **初始状态**.
5. $F \subseteq Q$ 是一个 **接受状态** 集合.

3.2 有穷自动机识别的语言

一个 DFA 可以识别某些语言. 给定一个输入字符串, 自动机从初始状态 q_0 开始, 根据输入字符串的每个符号和状态转移函数 δ 依次转移状态. 如果在处理完输入字符串后, 自动机停在一个接受状态 F 中的某个状态, 那么该字符串被认为是被该自动机接受的.

$$L(M) = \{w \in \Sigma^* \mid \text{the DFA} \quad M \quad \text{accepts} \quad w\} = \left\{w \in \Sigma^* \mid \hat{\delta}(q_0, w) \in F\right\} \tag{8}$$

这里面, $\hat{\delta}$ 是 δ 的扩展, 它定义了在给定初始状态和输入字符串的情况下, 自动机将转移到哪个状态.

3.3 有穷自动机的模拟运行

如果使用 Python 来模拟一个 DFA 的运行, 可以按照以下步骤进行:

```
class DFA:
    def __init__(self, s, a, t, q0, f):
        self.states = s  # 状态集合
        self.alphabet = a  # 输入符号集合
        self.transition = t  # 状态转移函数
        self.start_state = q0  # 初始状态
        self.accept_states = f  # 接受状态集合

    def run(self, input_string):
        current_state = self.start_state
        for symbol in input_string:
            if symbol in self.alphabet:
                current_state =
self.transition.get((current_state, symbol), None)
```

```
            if current_state is None:
                break
        else:
            break
    return current_state in self.accept_states
```

4 正则语言

4.1 计算的形式化定义

对于一个有穷自动机 $M = (Q, \Sigma, \delta, q_0, F)$ 和输入串 $w = w_1 w_2 ... w_n$, $w_i \in \Sigma$, 如果存在一个状态序列 $r_0, r_1, ..., r_n$, 使得

1. $r_0 = q_0$ (初始状态)
2. $r_{i+1} = \delta(r_i, w_{i+1})$ 对于 $0 \leq i < n$ (状态转移)
3. $r_n \in F$ (接受状态)

则称 M 接受输入串 w (接受计算). M 接受的所有字符串的集合称为 M 识别的语言, 记为 $L(M)$.

4.2 正则语言的定义

如果一个语言 L 中的所有字符串都可以被某个有穷自动机 M 接受, 则称 L 是一个 **正则语言 (Regular Language)**. 即存在一个有穷自动机 M 使得 $L = L(M)$.

4.3 非正则语言的例子

通常来说, 一个非正则语言 (即不能被任何有穷自动机识别的语言) 需要 **存储** 的能力 (requires memory). 例如, 语言 $L = \{a^n b^n \mid n \geq 0\}$ 包含所有形式为 a 的若干个后跟同样数量的 b 的字符串. 这个语言不是正则的, 因为有穷自动机无法记住它已经读了多少个 a, 以确保它读了相同数量的 b.

4.4 正则运算

正则语言在以下运算下是封闭的:

1. **并 (Union)**: 如果 L_1 和 L_2 是正则语言, 则 $L_1 \cup L_2$ 也是正则语言.
2. **连接(Concatenation)**: 如果 L_1 和 L_2 是正则语言, 则 $L_1 \circ L_2 = \{xy \mid x \in L_1, y \in L_2\}$ 也是正则语言.
3. **克林闭包 (Kleene Star)** : 如果 L 是正则语言, 则 $L^* = \{x_1 x_2 ... x_k \mid k \geq 0, x_i \in L\}$ 也是正则语言.

注意这里克林闭包是一元运算 (unary operation), 而并和连接是二元运算 (binary operations). 且克林闭包包含了空串 ($k = 0$ 的特殊情况), 即 $\varepsilon \in L^*$.

例子:

设 $A = \{\text{Good}, \text{Bad}\}$, $B = \{\text{Boy}, \text{Girl}\}$, 则

- $A \cup B = \{\text{Good}, \text{Bad}, \text{Boy}, \text{Girl}\}$
- $A \circ B = \{\text{GoodBoy}, \text{GoodGirl}, \text{BadBoy}, \text{BadGirl}\}$

- $A^* = \{\varepsilon, \text{Good}, \text{Bad}, \text{GoodGood}, \text{GoodBad}, \text{BadGood}, \text{BadBad}, \text{GoodGoodGood}, ...\}$

一些其他运算:

1. **补 (Complement)**: 如果 L 是正则语言, 则 $\overline{L} = \Sigma^* \setminus L$ 也是正则语言.
2. **交 (Intersection)**: 如果 L_1 和 L_2 是正则语言, 则 $L_1 \cap L_2$ 也是正则语言.
3. **差 (Difference)**: 如果 L_1 和 L_2 是正则语言, 则 $L_1 - L_2 = L_1 \cap \overline{L_2}$ 也是正则语言.
4. **对称差(Symmetric Difference)**: 如果 L_1 和 L_2 是正则语言, 则 $L_1 \oplus L_2 = (L_1 - L_2) \cup (L_2 - L_1) = (L_1 \cup L_2) - (L_1 \cap L_2)$ 也是正则语言.

5 非确定性有穷自动机

5.1 形式化定义

一个非确定性有穷自动机 (Nondeterministic Finite Automaton, NFA) 是一个五元组 $(Q, \Sigma, \delta, q_0, F)$, 其中

1. Q 是一个有限 **状态** 集合.
2. Σ 是一个有限 **输入符号** 集合, 称为 **字母表**.
3. $\delta : Q \times (\Sigma \cup \{\varepsilon\}) \to \mathcal{P}(Q)$ 是一个 **状态转移函数**, 其中 $\mathcal{P}(Q)$ 表示状态集合的幂集. 它定义了在给定当前状态和输入符号的情况下, 自动机可以转移到哪些状态. 注意这里允许 ε 转移, 即自动机可以在不读取任何输入符号的情况下转移状态.
4. $q_0 \in Q$ 是 **初始状态**.
5. $F \subseteq Q$ 是一个 **接受状态** 集合.

5.2 NFA 的状态转移

NFA 的状态转移函数 δ 定义了在给定当前状态和输入符号的情况下, 自动机可以转移到哪些状态. 具体来说, 对于每个状态 $q \in Q$ 和输入符号 $a \in \Sigma \cup \{\varepsilon\}$, $\delta(q, a)$ 是一个状态集合, 表示自动机可以从状态 q 通过读取输入符号 a 转移到的所有可能状态. 这意味着在某个状态下, 自动机可能有多个选择, 包括不读取任何输入符号而直接转移到另一个状态 (通过 ε 转移).

5.3 NFA 的计算过程

给定一个 NFA $M = (Q, \Sigma, \delta, q_0, F)$ 和输入串 $w = w_1 w_2 ... w_n$, $w_i \in \Sigma$, NFA 的计算过程可以描述如下:

1. **初始状态**: 计算从初始状态 q_0 开始, 包括所有通过 ε 转移可以到达的状态集合. 记为 $E(q_0)$.
2. **状态转移**: 对于输入串的每个符号 w_i (从 $i = 1$ 到 n), 计算当前状态集合 S_{i-1} (初始时为 $E(q_0)$) 通过读取符号 w_i 后可以到达的所有状态集合, 记为 S_i:

$$S_i = \bigcup_{q \in S_{i-1}} \delta(q, w_i) \tag{9}$$

然后, 计算 S_i 中所有状态通过 ε 转移可以到达的状态集合, 记为 $E(S_i)$.

3. **接受状态**: 在处理完输入串 w 后, 如果最终状态集合 S_n (包括通过 ε 转移可以到达的状态) 中包含至少一个接受状态 F 中的状态, 则称 NFA **接受** 输入串 w.

5.4 NFA 的 “猜想” 行为

NFA 的计算过程可以看作是对所有可能的状态路径进行 “猜测”. 在每个状态, NFA 可以选择多个可能的转移, 包括通过 ε 转移跳过某些输入符号. 这种非确定性使得 NFA 能够在某种程度上 “并行” 处理多个计算路径, 从而提高了对复杂语言的识别能力.

比如说, 如果我们要设计一个 NFA 来识别语言

$$L = \{x \mid x \text{ the third-to-last character of } x \text{ is } 1\} \tag{10}$$

且 $\Sigma = \{0, 1\}$, 我们可以设计如下的 NFA:

- 状态集合: $Q = \{q_0, q_1, q_2, q_3\}$
- 输入符号集合: $\Sigma = \{0, 1\}$
- 初始状态: q_0
- 接受状态: $F = \{q_3\}$
- 状态转移函数 δ:

当前状态	输入符号	下一个状态
q_0	0	q_0
q_0	1	q_0, q_1
q_1	0	q_2
q_1	1	q_2
q_2	0	q_3
q_2	1	q_3

可以看到, 在状态 q_0 读取到输入符号 `1` 时, NFA 可以选择留在 q_0 (继续读取更多的符号), 也可以转移到 q_1 (表示已经找到了一个可能的倒数第三个字符). 这种 “猜想” 行为使得 NFA 能够有效地识别符合条件的字符串.

如果我们使用 DFA 来识别同样的语言, 我们则必须要为最后三位可能的所有组合 $(000, 001, 010, 011, 100, 101, 110, 111)$ 设计状态, 这会导致状态数量的指数级增长.

5.5 ε-NFA 的转换

一个 ε-NFA 是一种特殊的 NFA, 它允许在不读取任何输入符号的情况下进行状态转移 (即通过 ε 转移). 这种能力使得 ε-NFA 在某些情况下更容易设计和理解.

我们可以通过以下步骤将一个 ε-NFA 转换为一个等价的 NFA:

1. **计算 ε-闭包**: 对于每个状态 $q \in Q$, 计算其 ε-闭包 $E(q)$, 即从状态 q 出发, 通过任意数量的 ε 转移可以到达的所有状态集合 (包含 q 自己).
2. **定义新的状态转移函数**: 对于每个状态 $q \in Q$ 和输入符号 $a \in \Sigma$, 定义新的状态转移函数 δ':

$$\delta'(q, a) = \bigcup_{p \in E(q)} \delta(p, a) \tag{11}$$

   这表示从状态 q 出发, 通过 ε 转移到达的所有状态 p,

然后读取输入符号 a 后可以到达的所有状态集合.

3. **定义新的接受状态**: 定义新的接受状态集合 F':

$$F' = \{q \in Q \mid E(q) \cap F \neq \emptyset\} \tag{12}$$

   这表示如果从状态 q 出发, 通过 ε 转移可以到达至少一个接受状态, 则 q 也是一个接受状态.
4. **构造新的 NFA**: 最终, 我们得到一个新的 NFA $M' = (Q, \Sigma, \delta', q_0, F')$, 它与原始的 ε-NFA 等价, 即它们识别相同的语言.

5.6 NFA 和 DFA 的等价性

如果两个自动机 M_1 和 M_2 识别相同的语言, 即 $L(M_1) = L(M_2)$, 则称它们是 **等价** 的.

DFA 显然是 NFA 的一个特例, 所以要证明 NFA 和 DFA 的等价性, 只需要证明对于任意一个 NFA, 都存在一个等价的 DFA.

我们可以通过以下步骤将一个 NFA 转换为一个等价的 DFA:

1. **状态集合**: 新的 DFA 的状态集合 Q' 是原始 NFA 的状态集合 Q 的幂集, 即 $Q' = \mathcal{P}(Q)$. 这意味着每个新的状态都是原始 NFA 的状态的一个子集.
2. **初始状态**: 新的 DFA 的初始状态 $q_{0'}$ 是原始 NFA 的初始状态 q_0 的 ε-闭包, 即 $q_{0'} = E(q_0)$.
3. **接受状态**: 新的 DFA 的接受状态集合 F' 包含所有包含至少一个原始 NFA 接受状态的子集, 即 $F' = \{S \subseteq Q \mid S \cap F \neq \emptyset\}$.
4. **状态转移函数**: 新的 DFA 的状态转移函数 δ' 定义如下:

$$\delta'(S, a) = \bigcup_{q \in S} \delta(q, a) \tag{13}$$

   这表示从状态集合 S 出发, 读取输入符号 a 后可以到达的所有状态集合.

通过上述步骤, 我们可以构造出一个新的 DFA $M' = (Q', \Sigma, \delta', q_{0'}, F')$, 它与原始的 NFA 等价, 即它们识别相同的语言.

推论: 一个语言是正则的, 当且仅当它可以被某个 NFA 识别. 即

$$L \text{ is regular } \Leftrightarrow \exists \text{ NFA } M \text{ s.t. } L = L(M) \tag{14}$$

子集构造法 (Subset Construction): 上述将 NFA 转换为 DFA 的方法称为子集构造法, 因为新的 DFA 的状态是原始 NFA 状态的子集.

6 正则表达式

6.1 引子

我们在算术中可以用运算符 (如 $+, -, \times, \div$) 来构造表达式, 类似地, 可以用正则运算符来构造描述语言的表达式, 称为正则表达式. 也就是说, 正则表达式的值是一个语言.

正则表达式能定义所有的正则语言, 反之亦然. 因此, 正则表达式和有穷自动机 (DFA/NFA) 是等价的.

6.2 正则表达式的形式化定义

6.2.1 归纳定义法

一个正则表达式 (Regular Expression, RE) 是通过以下规则递归定义的:

1. **基本表达式**:

- **(empty set)** $\emptyset$ 是一个正则表达式, 它表示空语言.
- **(empty string)** ε 是一个正则表达式, 它表示只包含空串的语言.
- **(literal character)** 对于每个符号 $a \in \Sigma$, a 是一个正则表达式, 它表示只包含字符串 a 的语言.

2. **复合表达式**:

- **(concatenation)** 如果 R_1 和 R_2 是正则表达式, 则 R_1R_2 是一个正则表达式, 它表示语言的连接 $L(R_1)L(R_2)$ (见 Section 1.1.4).
- **(alternation)** 如果 R_1 和 R_2 是正则表达式, 则 $R_1 + R_2$ 是一个正则表达式, 它表示语言的并 $L(R_1) \cup L(R_2)$ (见 Section 4.4).
- **(Kleene star)** 如果 R 是一个正则表达式, 则 R^* 是一个正则表达式, 它表示语言的克林闭包 $L(R)^*$ (见 Section 4.4).

正则表达式 R 所表示的语言记为 $L(R)$.

6.2.2 一些正则表达式的例子

假设字母表 $\Sigma = \{0, 1\}$, 则以下是一些正则表达式及其对应的语言:

1. $0^*10^* = \{w \mid w \text{ has exactly one } 1\}$. 这是因为 0^* 表示任意数量的 0 (包括零个), 因此 0^*10^* 表示一个 1 前后可以有任意数量的 0.

2. $\Sigma^*1\Sigma^* = \{w \mid w \text{ contains at least one } 1\}$.

3. $\Sigma^*001\Sigma^* = \{w \mid w \text{ contains 001 as a substring}\}$.

4. $1^*(01^+)^* = \{w \mid \text{every 0 in } w \text{ is followed by at least one 1}\}$. 注意这里为了方便起见, 我们用 R^+ 表示 RR^*.

5. $(\Sigma\Sigma)^* = \{w \mid \text{the length of } w \text{ is even}\}$.

6. $01 + 10 = \{01, 10\}$

7. $0\Sigma^*0 + 1\Sigma^*1 + 0 + 1 = \{w \mid w \text{ starts and ends with the same symbol}\}$

8. $(0 + \varepsilon)(1 + \varepsilon) = \{\varepsilon, 0, 1, 01\}$

9. $1^*\emptyset = \emptyset$. 注意空集连接任何语言仍然是空集.

10. $\emptyset^* = \{\varepsilon\}$. 星号运算把该语言中的任意个字符串连接在一起, 得到运算结果中的一个字符串. 如果该语言是空集, 星号运算能把 0 个字符串连接在一起, 结果就是空串.

6.3 运算的优先级

正则表达式中的运算符有不同的优先级, 其优先级从高到低依次为:

1. 克林闭 包 (*)
2. 连接
3. 并 (+)

6.4 有穷自动机和正则表达式

就描述语言的能力而言, 有穷自动机 (DFA/NFA) 和正则表达式是等价的. 具体来说, 任何的正则表达式都能够转换成能识别它所描述语言的有穷自动机, 反之亦然.

6.4.1 如果一个语言可以用正则表达式描述, 那么它是正则的.

考虑如下的证明思路:

1. 由于 NFA 和 DFA 是等价的, 因此只需要证明对于任意一个正则表达式 R, 都存在一个等价的 NFA M 使得 $L(R) = L(M)$.

2. 对于每个基本正则表达式, 我们可以直接构造一个等价的 NFA:
   - 对于 $\emptyset$, 构造一个 NFA $M = (\{q_0\}, \Sigma, \delta, q_0, \emptyset)$, 其中 δ 是一个空函数. 该 NFA 没有接受状态, 因此它识别的语言是空集.
   - 对于 ε, 构造一个 NFA $M = (\{q_0\}, \Sigma, \delta, q_0, \{q_0\})$, 其中 δ 是一个空函数. 该 NFA 的初始状态也是接受状态, 因此它识别的语言只包含空串.

- 对于每个符号 $a \in \Sigma$, 构造一个 NFA $M = (\{q_0, q_1\}, \Sigma, \delta, q_0, \{q_1\})$, 其中 $\delta(q_0, a) = \{q_1\}$ 且 δ 在其他情况下是空函数. 该 NFA 识别的语言只包含字符串 a.

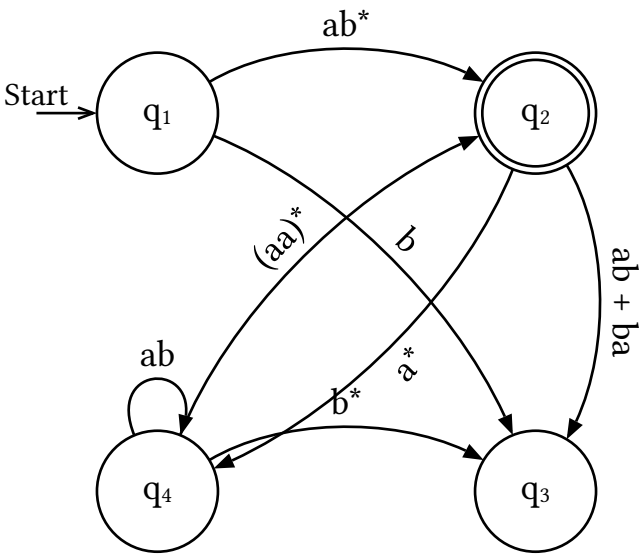
3. 对于复合正则表达式, 我们可以通过归纳假设来构造等价的 NFA:
   - 对于 R_1R_2, 假设存在等价的 NFA M_1 和 M_2 分别识别 R_1 和 R_2. 我们可以构造一个新的 NFA M, 通过将 M_1 的所有接受状态与 M_2 的初始状态通过 ε 转移连接起来. 这样, M 识别的语言就是 $L(R_1)L(R_2)$.
   - 对于 $R_1 + R_2$, 假设存在等价的 NFA M_1 和 M_2 分别识别 R_1 和 R_2. 我们可以构造一个新的 NFA M, 通过添加一个新的初始状态 $q_{0'}$ 和两个 ε 转移, 分别从 $q_{0'}$ 指向 M_1 和 M_2 的初始状态. 这样, M 识别的语言就是 $L(R_1) \cup L(R_2)$.
   - 对于 R^*, 假设存在一个等价的 NFA M 识别 R. 我们可以构造一个新的 NFA M', 通过添加一个新的初始状态 $q_{0'}$ 和一个新的接受状态 $q_{f'}$. 然后, 添加两个 ε 转移, 一个从 $q_{0'}$ 指向 M 的初始状态, 另一个从 M 的所有接受状态指向 $q_{0'}$. 另外, 添加一个 ε 转移从 $q_{0'}$ 指向 $q_{f'}$, 以允许接受空串. 这样, M' 识别的语言就是 $L(R)^*$.

4. 通过上述步骤, 我们可以递归地构造出一个等价的 NFA M 对应于任意的正则表达式 R, 从而证明了如果一个语言可以用正则表达式描述, 那么它是正则的.

6.4.2 如果一个语言是正则的, 则可以用正则表达式描述它

考虑如下的证明思路:

1. 由于 NFA 和 DFA 是等价的, 因此只需要证明对于任意一个 DFA M, 都存在一个等价的正则表达式 R 使得 $L(R) = L(M)$.
2. 该过程需要引入一种称为 **广义非确定型有穷自动机** (Generalized Nondeterministic Finite Automaton, GNFA) 的中间模型. GNFA 与 NFA 类似, 但它的状态转移函数允许使用正则表达式作为标签, 而不仅仅是单个符号或 ε. 如下图所示:



3. GNFA 还满足以下条件:
   - GNFA 只有一个初始状态, 它有到所有其他状态的转移, 但没有任何进入它的转移.
   - GNFA 只有一个接受状态, 它有从所有其他状态的转移, 但没有任何离开它的转移.
   - 除了初始状态和接受状态之外, 每对状态之间恰好有一条单向转移 (可能标签为 $\emptyset$).
4. 考虑 DFA $\to$ GNFA 的构造过程: 给定一个 DFA $M = (Q, \Sigma, \delta, q_0, F)$, 我们可以构造一个等价的 GNFA $M' = \left(Q', \Sigma, \delta', q_{0'}, q_{f'}\right)$, 其中:
   - 状态集合: $Q' = Q \cup \left\{q_{0'}, q_{f'}\right\}$, 其中 $q_{0'}$ 是新的初始状态, $q_{f'}$ 是新的接受状态.
   - 输入符号集合: Σ 与原始 DFA 相同.
   - 状态转移函数 δ' 定义如下:
     - 对于每个状态 $q \in Q$, 添加一个从 $q_{0'}$ 到 q 的转移, 标签为 ε.
     - 对于每个接受状态 $f \in F$, 添加一个从 f 到 $q_{f'}$ 的转移, 标签为 ε.
     - 对于每对状态 $(p, q) \in Q \times Q$, 如果存在一个输入符号 $a \in \Sigma$ 使得 $\delta(p, a) = q$, 则添加一个从 p 到 q 的转移, 标签为 a; 如果有多个这样的输入符号, 则标签为它们的并 (用 $+$ 连接); 如果没有这样的输入符号, 则标签为 $\emptyset$.
5. 考虑 GNFA $\to$ 正则表达式 的转换过程: 给定一个 GNFA $M' = \left(Q', \Sigma, \delta', q_{0'}, q_{f'}\right)$, 我们可以通过逐步消除状态来构造一个等价的正则表达式 R:
   - 选择一个非初始状态且非接受状态 $r \in Q' \setminus \left\{q_{0'}, q_{f'}\right\}$.
   - 对于每对状态 $(p, q) \in Q' \times Q'$ (包括 $p = r$ 或 $q = r$), 更新状态转移函数 δ' 以反映消除状态 r 的影响:

    - 如果存在从 p 到 r 的转移, 标签为 R_1, 从 r 到 r 的转移, 标签为 R_2, 以及从 r 到 q 的转移, 标签为 R_3, 则添加一个从 p 到 q 的新转移, 标签为 $R_1R_2^*R_3$.
    - 如果存在从 p 到 r 的转移, 标签为 R_1, 以及从 r 到 q 的转移, 标签为 R_3, 则添加一个从 p 到 q 的新转移, 标签为 R_1R_3.
    - 如果存在从 p 到 q 的转移, 标签为 R_4, 则更新该转移的标签为 $R_4 + R_1R_2^*R_3$ (如果同时存在上述情况).
  - 删除状态 r 及其所有相关的转移.
  - 重复上述步骤直到只剩下初始状态和接受状态.
6. 最终, 当只剩下初始状态 $q_{0'}$ 和接受状态 $q_{f'}$ 时, 状态转移函数 δ' 中从 $q_{0'}$ 到 $q_{f'}$ 的转移标签就是所求的正则表达式 R.

6.5 正则表达式代数定律

6.5.1 正则表达式的等价性

如果两个正则表达式定义了相同的语言, 则称这两个正则表达式是等价的.

例如, 下面两个表达式都表示交替的 0 和 1 的串的集合

- $(\varepsilon + 1)(01)^*(\varepsilon + 0)$
- $(01)^* + (10)^* + 1(01)^* + 0(10)^*$

6.5.2 单位元和零元

1. 单位元: $R + \emptyset = R$, $R\varepsilon = \varepsilon R = R$
2. 零元: $R\emptyset = \emptyset R = \emptyset$, $R + \Sigma^* = \Sigma^*$

6.5.3 正则表达式的代数定律

1. 交换律: $R_1 + R_2 = R_2 + R_1$
2. 结合律: $(R_1 + R_2) + R_3 = R_1 + (R_2 + R_3)$, $(R_1R_2)R_3 = R_1(R_2R_3)$
3. 分配律: $R_1(R_2 + R_3) = R_1R_2 + R_1R_3$, $(R_1 + R_2)R_3 = R_1R_3 + R_2R_3$

6.5.4 幂等律

1. $R + R = R$
2. $R \cap R = R$

6.5.5 克林闭包的性质

1. $\varepsilon \in L(R^*)$
2. $(R^*)^* = R^*$
3. $\emptyset^* = \varepsilon$, $\varepsilon^* = \varepsilon$
4. $R^+ = RR^* = R^*R$

5. $R^* = R^+ + \varepsilon$

6.6 Arden 引理

6.6.1 内容

设 P 和 Q 是正则表达式, 则关于正则表达式 X 的方程 $X = Q + XP$ 的解为 $X = QP^*$. 如果 $\varepsilon \notin L(P)$, 则该解是唯一的.

6.6.2 使用 Arden 引理把 NFA 转换为正则表达式

前提假设:

1. 有穷自动机的状态转移图中不包含 ε 转移.
2. 有穷自动机仅含有一个初始状态

转换步骤:

1. 对于有穷自动机的每个状态 q_i, 定义一个正则表达式 R_i 表示从初始状态出发到达状态 q_i 的所有字符串的集合.
2. 根据状态转移图, 对每个状态 q_i, 写出一个关于 R_i 的方程. 该方程的形式为:

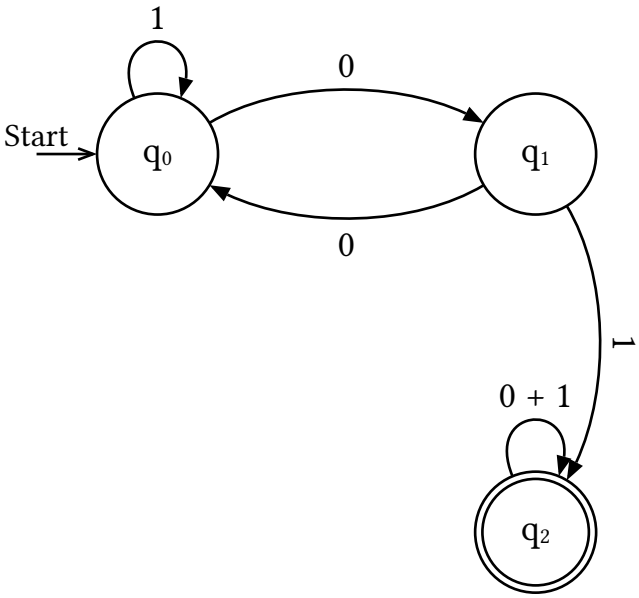
$$R_i = \sum_j R_j a_{ji} + b_i \tag{15}$$

其中, a_{ji} 是从状态 q_j 到状态 q_i 的输入符号 (如果存在多条边, 则用 $+$ 连接), b_i 是从初始状态直接到达状态 q_i 的输入符号 (如果存在多条边, 则用 $+$ 连接; 如果没有, 则为 $\emptyset$).

3. 对于每个接受状态 q_f, 使用 Arden 引理求解关于 R_f 的方程, 得到 R_f 的正则表达式.
4. 最终, 语言 $L(M)$ 可以表示为所有接受状态的正则表达式的并:

$$L(M) = \sum_{q_f \in F} R_f \tag{16}$$

例子: 考虑如下的 DFA:



1. 定义正则表达式: R_0, R_1, R_2 分别表示从初始状态 q_0 出发到达状态 q_0, q_1, q_2 的所有字符串的集合.
2. 写出方程:
   - 对于状态 q_0: $R_0 = R_0 1 + R_1 0 + \varepsilon$
   - 对于状态 q_1: $R_1 = R_0 0$
   - 对于状态 q_2: $R_2 = R_1 1 + R_2 0 + R_2 1$
3. 求解方程:
   - 从第二个方程得到 $R_1 = R_0 0$.
   - 将 R_1 代入第一个方程, 得到 $R_0 = R_0 1 + R_0 00 + \varepsilon = R_0(1 + 00) + \varepsilon$. 根据 Arden 引理, 得到 $R_0 = \varepsilon(1 + 00)^* = (1 + 00)^*$.
   - 将 R_1 代入第三个方程, 得到 $R_2 = R_0 01 + R_2(0 + 1)$. 根据 Arden 引理, 得到 $R_2 = R_0 01(0 + 1)^* = (1 + 00)^* 01(0 + 1)^*$.
4. 语言 $L(M)$ 可以表示为 R_2: $L(M) = R_2 = (1 + 00)^* 01(0 + 1)^*$.

6.7 正则表达式的应用

6.7.1 `grep` 工具

`grep` 是一个在类 Unix 系统中广泛使用的命令行工具, 用于在文件中搜索符合特定模式的文本行.

例子:

- 搜索包含 “error” 的行: `grep "error" filename.txt`
- 使用正则表达式搜索以 “a” 开头, 后跟任意数量的字符, 然后是 “b” 的行: `grep "^a.*b$" filename.txt`
- 忽略大小写搜索 “warning”: `grep -i "warning" filename.txt`

7 泵引理 (Pumping Lemma)

7.1 引子

DFA 的状态数量是有限的, 因此当输入字符串足够长时, DFA 必须在某些状态之间循环 (即鸽巢原理), 而重复路径对应的子串就可以任意重复或删除而不影响字符串是否被接受. 泵引理正是基于这一观察, 它为正则语言提供了一个 **必要条件**.

7.2 引理内容

设 L 是一个正则语言, 则存在一个整数 $p \geq 1$ (称为泵长度), 使得对于任意字符串 $s \in L$ 且 $|s| \geq p$, 都可以将 s 分割为三个子串 $s = xyz$, 满足以下条件:

1. $|y| \geq 1$ (即子串 y 非空)
2. $|xy| \leq p$ (即子串 xy 的长度不超过泵长度 p)
3. 对于所有整数 $i \geq 0$, 字符串 $xy^iz \in L$ (即通过重复子串 y 任意次数, 生成的新字符串仍然属于语言 L)

7.3 证明思路

设 L 是一个正则语言, 则存在一个识别 L 的 DFA $M = (Q, \Sigma, \delta, q_0, F)$, 其中 $|Q| = n$ (状态数量). 取泵长度 $p = n$. 对于任意字符串 $s \in L$ 且 $|s| \geq p$, 在处理字符串 s 时, DFA 必须经过至少 $p + 1$ 个状态 (包括初始状态). 根据鸽巢原理, 在这 $p + 1$ 个状态中, 至少有两个状态是相同的. 设这两个相同的状态分别在处理字符串 s 的第 k 个和第 m 个字符后到达, 其中 $0 \leq k < m \leq |s|$. 将字符串 s 分割为三个子串:

- $x = s[0 : k]$ (从第 0 个字符到第 $k - 1$ 个字符)
- $y = s[k : m]$ (从第 k 个字符到第 $m - 1$ 个字符)
- $z = s[m : |s|]$ (从第 m 个字符到最后一个字符)

由于在处理子串 y 时, DFA 从状态 q 回到状态 q, 因此对于任意整数 $i \geq 0$, 处理字符串 xy^iz 时, DFA 仍然会到达接受状态 (因为它与处理字符串 s 时经过的状态路径相同). 因此, $xy^iz \in L$. 这证明了泵引理的内容.

7.4 使用泵引理证明语言非正则

要使用泵引理证明一个语言 L 非正则, 我们可以采用反证法. 假设 L 是正则的, 则根据泵引理, 存在一个泵长度 $p \geq 1$. 然后, 我们需要找到一个字符串 $s \in L$ 且 $|s| \geq p$, 使得对于任意的分割 $s = xyz$ 满足泵引理的条件, 都存在一个整数 $i \geq 0$ 使得 $xy^iz\neg \in L$. 这就与泵引理的结论矛盾, 因此 L 非正则.

8 极小化 DFA

8.1 引子

极小化 DFA 的目标是构造一个状态数量最少的 DFA, 该 DFA 与原始 DFA 识别相同的语言.

8.2 等价状态

一个 DFA 的两个状态 q_i 和 q_j 被称为是 **等价** 的, 如果对于任意输入字符串 $w \in \Sigma^*$, 从状态 q_i 出发处理字符串 w 和从状态 q_j 出发处理字符串 w 最终都到达相同的接受状态类别 (即要么都到达接受状态, 要么都不到达接受状态).

特别地, 当 $|w| = 0$ (即 $w = \varepsilon$) 时, 如果 q_i 和 q_j 是等价的, 则它们要么都属于接受状态集合 F, 要么都不属于 F. 这时候我们称它们是 0-等价的.

以此类推, 如果对于任意输入字符串 $w \in \Sigma^*$ 且 $|w| = k$, 从状态 q_i 出发处理字符串 w 和从状态 q_j 出发处理字符串 w 最终都到达相同的接受状态类别, 则称它们是 **k-等价** 的.

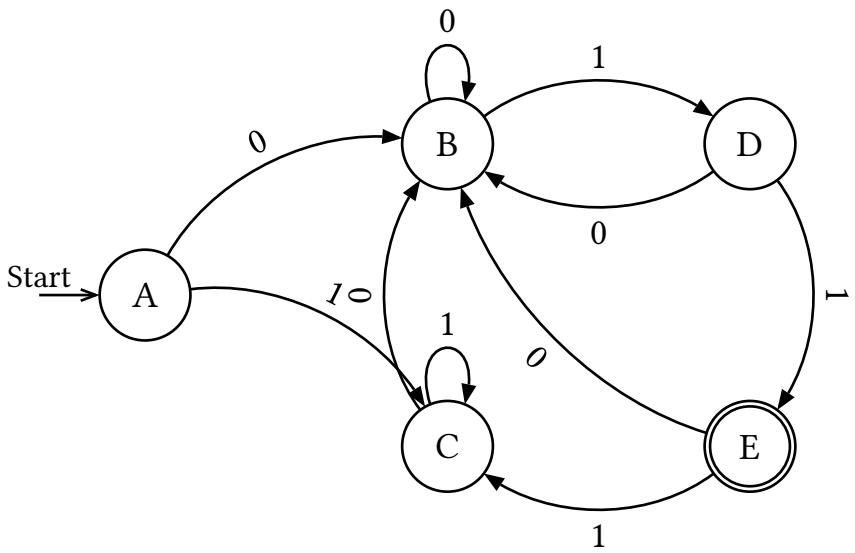
状态的可区分性: 如果存在一个输入字符串 $w \in \Sigma^*$, 使得从状态 q_i 出发处理字符串 w 和从状态 q_j 出发处理字符串 w 最终到达不同的接受状态类别, 则称状态 q_i 和 q_j 是 **可区分** 的. **不可区分** 的状态即为等价状态.

8.3 划分等价类法

基本思想:

1. 将 DFA 的状态集切分成等价状态类
2. 从每个等价状态类中选取一个代表, 构造新的极小化 DFA

例如对于如下 DFA:



1. 划分 0-等价类:

- 接受状态类: $F = \{E\}$
- 非接受状态类: $Q \setminus F = \{A, B, C, D\}$

2. 划分 1-等价类:

- 状态 A, B 和 C 属于同一类, 因为它们在输入符号 `0` 和 `1` 下都转移到非接受状态类.
- 状态 D 属于单独一类, 因为它在输入符号 `1` 下转移到接受状态类.
- 因此, 1-等价类为: $\{A, B, C\}$ 和 $\{D\}$ 以及 $\{E\}$.

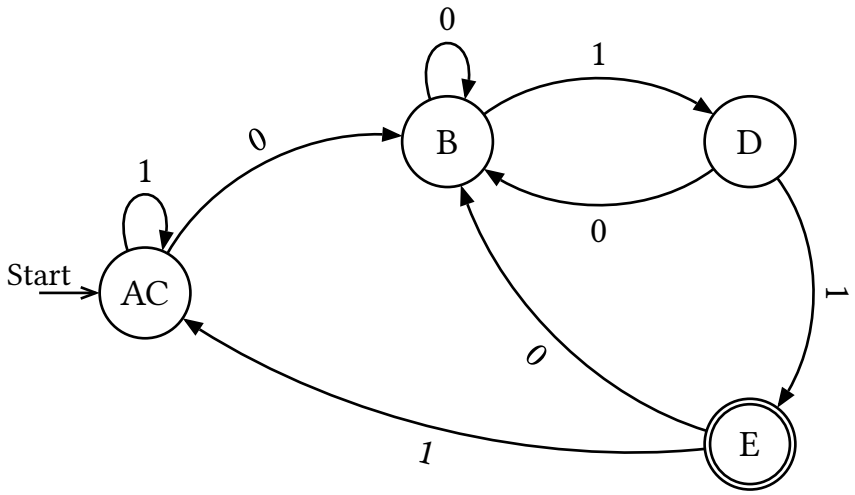
3. 划分 2-等价类:

- 状态 A 和 C 属于同一类, 状态 B 属于单独一类, 状态 D 和 E 仍然分别属于各自的类.
- 因此, 2-等价类为: $\{A, C\}$ 和 $\{B\}$ 以及 $\{D\}$ 和 $\{E\}$.

4. 划分 3-等价类:

- 划分结果与 2-等价类相同, 因为 A, C 无法在长度为 3 的输入字符串下区分开来.

5. 构造极小化 DFA:



存在不可达状态时的处理: 在极小化 DFA 之前, 需要先移除所有不可达状态. 不可达状态是指从初始状态出发无法到达的状态. 这些状态不会影响 DFA 识别的语言, 因此可以安全地删除它们以简化 DFA.

8.4 Myhill-Nerode 定理

8.4.1 字符串的语言不可区分性

设 A 是一个语言, 则对于任意两个字符串 $x, y \in \Sigma^*$, 如果对于所有字符串 $z \in \Sigma^*$, 字符串 $xz \in A$ 当且仅当 $yz \in A$, 则称字符串 x 和 y 是 **不可区分** 的 (with respect to A). 否则, 它们是 **可区分** 的.

直观来讲, 如果从字符串 x 和 y 出发, 无论后续添加什么字符串 z, 它们是否属于语言 A 的结果总是相同的, 则称它们是不可区分的.

如果 x 和 y 是用 A 不可区分的, 则记为 $x \underset{A}{\equiv} y$. 这定义了一个等价关系, 因为它满足自反性, 对称性和传递性. 通过该等价关系, 我们可以将字符串集合 Σ^* 划分为若干个等价类, 每个等价类包含所有彼此不可区分的字符串:

$$\Sigma^* = \bigcup_{i \in I} [x_i]_A \tag{17}$$

其中

$$[x_i]_A = \left\{ y \in \Sigma^* \mid y \underset{A}{\equiv} x_i \right\} \tag{18}$$

这个等价类中, 如果有一个字符串能被语言 A 接受, 则该等价类中的所有字符串都能被接受, 因为根据不可区分的定义, 我们可以取 $z = \varepsilon$.

8.4.2 字符串的 DFA 不可区分性

设 $M = (Q, \Sigma, \delta, q_0, F)$ 是一个 DFA, 则对于任意两个字符串 $x, y \in \Sigma^*$, 如果 $\delta^*(q_0, x) = \delta^*(q_0, y)$, 则称字符串 x 和 y 是 **不可区分** 的 (with respect to M). 否则, 它们是 **可区分** 的.

这里 δ^* 是状态转移函数 δ 的扩展, 它定义如下:

- $\delta^*(q, \varepsilon) = q$;
- $\delta^*(q, aw) = \delta^*(\delta(q, a), w)$, 其中 $a \in \Sigma$.

直观来讲, 如果从初始状态 q_0 出发, 处理字符串 x 和 y 最终到达相同的状态, 则称它们是不可区分的.

类似地我们也可以用字符串的 DFA 不可区分性定义一个等价关系 $\underset{M}{\equiv}$, 并将字符串集合 Σ^* 划分为若干个等价类.

8.4.3 两种等价类的关系

设 $A = L(M)$, 则对于任意字符串 $x, y \in \Sigma^*$, 有

$$x \underset{M}{\equiv} y \Longrightarrow x \underset{A}{\equiv} y \tag{19}$$

这告诉我们, M 对 Σ^* 的划分比 A 对 Σ^* 的划分更细致 (即 M 的等价类数量不少于 A 的等价类数量).

推论: 如果语言 A 是正则的, 那么 $\underset{A}{\equiv}$ 的等价类的数目是有限的. 这是因为如果 A 是正则的, 则存在一个识别 A 的 DFA M, 而 DFA 的状态数量是有限的, 因此 $\underset{M}{\equiv}$ 的等价类数量也是有限的. 由于 $\underset{A}{\equiv}$ 的等价类数量不多于 $\underset{M}{\equiv}$ 的等价类数量, 因此 $\underset{A}{\equiv}$ 的等价类数量也是有限的.

8.4.4 Myhill-Nerode Theorem

1. 语言 A 是正则的, 当且仅当 $\underset{A}{\equiv}$ 的等价类的数量是有限的.
2. 如果语言 A 是正则的, 则存在一个识别 A 的极小化 DFA, 其状态数量等于 $\underset{A}{\equiv}$ 的等价类的数量.

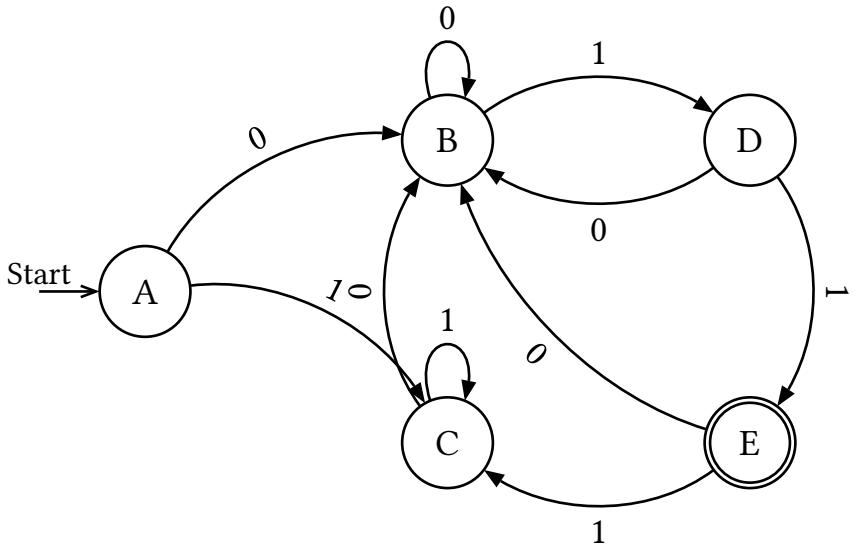
8.4.5 填表法

填表法是一种基于 Myhill-Nerode 定理的 DFA 极小化算法. 其基本思想是通过识别和合并不可区分的状态来减少 DFA 的状态数量.

算法步骤:

1. 初始化: 创建一个表格, 其中行和列分别对应 DFA 的状态. 将表格的下三角部分初始化为空, 因为我们只需要考虑每对状态一次.
2. 标记接受状态和非接受状态: 对于每对状态 (q_i, q_j), 如果一个是接受状态而另一个不是, 则将表格中的对应单元格标记为可区分 (通常用 `X` 表示).
3. 迭代标记: 重复以下步骤直到没有新的标记:
   - 对于每对未标记的状态 (q_i, q_j), 检查对于每个输入符号 $a \in \Sigma$, 它们的转移状态 $\delta(q_i, a)$ 和 $\delta(q_j, a)$ 是否已经被标记为可区分.
   - 如果存在一个输入符号 a 使得转移状态被标记为可区分, 则将 (q_i, q_j) 标记为可区分.
4. 合并不可区分的状态: 对于所有未标记的状态对 (q_i, q_j), 将它们合并为一个状态, 因为它们是不可区分的.
5. 构造极小化 DFA: 使用合并后的状态构造新的 DFA, 并更新状态转移函数和接受状态集合.

例子:



1. 初始化表格:

	A	B	C	D	E
A					
B					
C					
D					
E					

2. 标记接受状态和非接受状态对:

	A	B	C	D	E
A					
B					
C					
D					
E	X	X	X	X	

3. 迭代标记:

第一轮: 观察到现在只有 (C, E) 这样的状态是可以区分的, 所以我们只考虑能通过一次转移到达 E 的状态对, 这里能转移到 E 的状态只有 D:

- 检查 (A, D), 对于输入 `1`, $\delta(A, 1) = C$ 和 $\delta(D, 1) = E$, 因此标记 (A, D) 为可区分.
- 检查 (B, D), 对于输入 `1`, $\delta(B, 1) = D$ 和 $\delta(D, 1) = E$, 因此标记 (B, D) 为可区分.
- 检查 (C, D), 对于输入 `1`, $\delta(C, 1) = C$ 和 $\delta(D, 1) = E$, 因此标记 (C, D) 为可区分.

	A	B	C	D	E
A					
B					
C					
D	X	X	X		
E	X	X	X	X	

第二轮: 观察到现在 (A, D), (B, D) 和 (C, D) 都是可区分的, 所以我们只考虑能通过一次转移到达 D 的状态对, 这里能转移到 D 的状态只有 B:

- 检查 (A, B), 对于输入 `1`, $\delta(A, 1) = C$ 和 $\delta(B, 1) = D$, 因此标记 (A, B) 为可区分.
- 检查 (B, C), 对于输入 `1`, $\delta(B, 1) = D$ 和 $\delta(C, 1) = C$, 因此标记 (B, C) 为可区分.

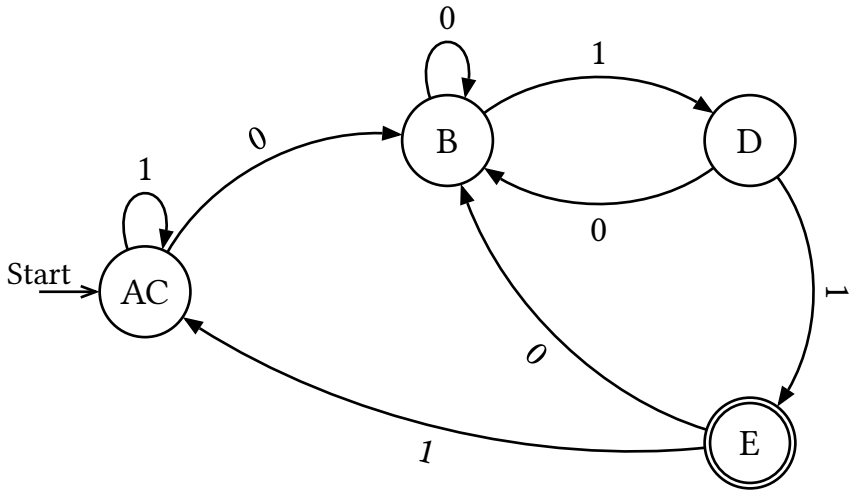
	A	B	C	D	E
A					
B	X				
C		X			
D	X	X	X		
E	X	X	X	X	

第三轮: 还有未标记的状态对 (A, C), 检查 (A, C), 对于输入 `0`, $\delta(A, 0) = B$ 和 $\delta(C, 0) = B$, 对于输入 `1`, $\delta(A, 1) = C$ 和 $\delta(C, 1) = C$, 因此 (A, C) 仍然是不可区分的.

4. 合并不可区分的状态:

将状态 A 和 C 合并为一个状态 AC.

5. 构造极小化 DFA:



8.5 判断 DFA 的等价性

8.5.1 方法一

将两个 DFA 极小化, 然后比较它们的状态转移图是否同构 (isomorphic). 如果同构, 则两个 DFA 等价; 否则不等价.

同构: 存在一个双射映射 $f : Q_1 \to Q_2$, 使得对于任意状态 $q \in Q_1$ 和输入符号 $a \in \Sigma$, 都有 $f(\delta_1(q, a)) = \delta_2(f(q), a)$, 且 $q \in F_1$ 当且仅当 $f(q) \in F_2$.

8.5.2 方法二

令 M 和 $\widetilde{M}$ 是两个 DFA, Σ 是输入符号集合.

1. 构造一个状态为 $\{q, \tilde{q}\}$ 的 DFA N, 其中 q 是 M 的状态, $\tilde{q}$ 是 $\widetilde{M}$ 的状态, 写出它的状态转移表.
2. 如果在某一步中, 我们发现一个状态对 $\{q, \tilde{q}\}$ 中一个是接受状态而另一个不是, 则 M 和 $\widetilde{M}$ 不等价.
3. 如果我们遍历了所有可能的状态对而没有发现上述情况,则 M 和 $\widetilde{M}$ 等价.

9 文法 (Grammar)

9.1 引子

我们如何描述语言?

1. 自动机: 通过状态转移图描述语言的生成过程.
2. 正则表达式: 通过代数表达式描述语言的结构.
3. 文法: 语言的 **有穷描述**, 通过一组生成规则描述语言的构造方式.

当我们描述一种语言时, 就是要说明这种语言的句子

- 如果语言只含有有穷多个句子, 则可以直接列举出所有句子.
- 如果语言含有无穷多个句子, 存在着如何给出它的有穷表示的问题. 这需要一种规则, 用这些规则来描述语言的结构, 可以把这些规则看成一种元语言，这些规则就称为文法.

> A *language* is a collection of sentences of finite length all constructed from a finite alphabet of symbols.
>
> A *grammar* can be regarded as a device that enumerates the sentences of a language.
>
> — Noam Chomsky

9.2 形式化定义

文法 (Grammar) G 是一个四元组 (V, T, P, S), 其中:

- V 是一个有限的符号集合. $A \in V$ 称为 **变量** (Variables) 或 **非终结符** (Nonterminals), 表示一个 **语法范畴** (Syntactic Categories).
- T 是 **终结符** (Terminals) 的有限集合, 且 $V \cap T = \emptyset$. 终结符是语言的基本符号, 例如字母, 数字, 标点符号等. 语言的句子由终结符组成.
- $S \in V$ 是 **开始符号** (Start Symbol), 表示语言的起始范畴.
- P 是 **产生式** (Productions) 的有限集合, 每个产生式的形式为 $\alpha \to \beta$, 其中 $\alpha \in (V \cup T)^+$, $\beta \in (V \cup T)^*$. 产生式定义了如何从变量生成终结符和其他变量的组合.

约定

1. 对一组有相同左部的产生式 $\alpha \to \beta_1, \alpha \to \beta_2, \ldots, \alpha \to \beta_n$, 我们可以将它们简写为 $\alpha \to \beta_1 \mid \beta_2 \mid \ldots \mid \beta_n$. 这里 $\beta_1, \beta_2, \ldots, \beta_n$ 被称为 **候选** (Candidates).
2. 使用符号:

- 英文字母表较为前面的大写字母 (如 $A, B, C, \ldots$) 表示变量.

- 英文字母表较为后面的小写字母 (如 $a, b, c, \ldots$) 表示终结符.
- 希腊字母 (如 $\alpha, \beta, \gamma, \ldots$) 表示变量和终结符的混合.

文法的例子

考虑一个文法 $G = (V, T, P, S)$ 用于生成所有由偶数个 a 组成的字符串, 其中

- $V = \{S\}, T = \{a\}, S = S, P = \{S \to aaS \mid \varepsilon\}$

该文法通过 $S \to aaS$ 产生偶数个 a, 通过 $S \to \varepsilon$ 终止生成过程. 该文法可以生成字符串 $\varepsilon, aa, aaaa, aaaaaa, \ldots$.

9.3 推导 (Derivation)

在文法 G 中, 如果存在一个产生式 $\alpha \to \beta$ 且字符串 w 可以表示为 $w = x\alpha y$ (其中 $x, y \in (V \cup T)^*$), 则我们可以通过将 α 替换为 β 来得到字符串 $w' = x\beta y$. 这种替换称为 **直接推导**, 记为 $w \underset{G}{\Rightarrow} w'$.

另一方面, 如果 $w \underset{G}{\Rightarrow} w'$, 称 w' 在文法 G 中直接 **规约** (Reduce) 为 w.

基于此, 我们可以定义

1. $w \underset{G^n}{\Rightarrow} w'$: 如果存在一系列字符串 $w_0, w_1, w_2, \ldots, w_n$ 使得 $w_0 = w$, $w_n = w'$, 且对于每个 i (其中 $0 \le i < n$), 都有 $w_i \underset{G}{\Rightarrow} w_{i+1}$, 则称 w' 在文法 G 中经过 n 步推导得到 w, 记为 $w \underset{G^n}{\Rightarrow} w'$.
2. $w \underset{G^*}{\Rightarrow} w'$: 如果存在一个整数 $n \ge 0$ 使得 $w \underset{G^n}{\Rightarrow} w'$, 则称 w' 在文法 G 中推导得到 w, 记为 $w \underset{G^*}{\Rightarrow} w'$.
3. $w \underset{G^+}{\Rightarrow} w'$: 如果存在一个整数 $n \ge 1$ 使得 $w \underset{G^n}{\Rightarrow} w'$, 则称 w' 在文法 G 中经过至少一步推导得到 w, 记为 $w \underset{G^+}{\Rightarrow} w'$.

9.4 语言, 句子和句型

设文法 $G = (V, T, P, S)$, 则由 G 生成的语言 $L(G)$ 定义为:

$$L(G) = \left\{ w \in T^* \mid S \underset{G^*}{\Rightarrow} w \right\} \tag{20}$$

其中 w 称为语言 $L(G)$ 的 **句子** (Sentences) 或 **字符串** (Strings).

此外, 如果存在一个字符串 $w \in (V \cup T)^*$ 使得 $S \underset{G^*}{\Rightarrow} w$, 则称 w 是文法 G 的 **句型** (Sentential Forms).

注: 句型可以包含变量和终结符, 而句子仅包含终结符.

9.5 文法的构造

9.5.1 凑规则

给定一个语言, 我们需要找到一套规则. 这套规则能生成这个语言里的所有句子, 不多也不少. 一种常用的方法是 **凑规则**. 这不是一个严格的算法, 而是一种启发式的方法, 通过分析语言的结构和模式, 来设计合适的产生式.

1. 找出语言的若干句子
2. 分析句子的特点
3. 根据句子的特点凑规则
4. 验证规则的正确性

9.5.2 文法的等价

如果两个文法 G_1 和 G_2 生成相同的语言, 即 $L(G_1) = L(G_2)$, 则称它们是 **等价** 的.

9.5.3 例子

构造文法 G, 使得 $G = \{0, 1, 00, 11\}$

将文法的开始符号定义为这 4 个句子

$$G_1 = (\{S\}, \{0, 1\}, \{S \to 0 \mid 1 \mid 00 \mid 11\}, S) \tag{21}$$

用变量 A 和 B 分别表示单个 0 和单个 1

$$G_2 = (\{S, A, B\}, \{0, 1\}, \{S \to A \mid B \mid AA \mid BB, A \to 0, B \to 1\}, S) \tag{22}$$

$L = \{0^{2n}1^{3n} \mid n \geq 0\}$

$$G_3 = (\{S\}, \{0, 1\}, \{S \to \varepsilon \mid 00S111\}, S) \tag{23}$$

$L = \{ww^T \mid w \in \{0, 1, 2, 3\}^+\}$, 其中 w^T 是字符串 w 的反转 (reverse)

可以利用递归的方法来构造文法:

$$\begin{aligned} G &= (V, T, P, S) \\ V &= \{S\} \\ T &= \{0, 1, 2, 3\} \\ P &= \{S \to 0S0 \mid 1S1 \mid 2S2 \mid 3S3 \mid 00 \mid 11 \mid 22 \mid 33\} \end{aligned} \tag{24}$$

$L = \{w \mid w \text{ is decimal rational number}\}$

$$
\begin{aligned}
G &= (V, T, P, S) \\
V &= \{S, R, N, M, A, B, D\} \\
T &= \{0, 1, 2, 3, 4, 5, 6, 7, 8, 9, ., \text{-}, +\} \\
P &= \{S \to R \mid \text{-}R \mid +R, \\
&\quad R \to N \mid B, \\
&\quad B \to N.D, \\
&\quad N \to 0 \mid AM, \\
&\quad D \to 0 \mid MA, \\
&\quad A \to 1 \mid 2 \mid 3 \mid 4 \mid 5 \mid 6 \mid 7 \mid 8 \mid 9, \\
&\quad M \to A \mid AM \mid \varepsilon \\
&\}
\end{aligned} \tag{25}
$$

这里

- S 表示整个数字
- R 表示可选的符号和数字部分
- N 表示整数部分, 如果超过 0 则不能以 0 开头
- B 表示带小数点的数字
- D 表示小数部分, 不能以 0 结尾
- A 表示非零数字
- M 表示任意数量的数字

9.6 正则文法

9.6.1 文法的 Chomsky 体系

设 $G = (V, T, P, S)$ 是一个文法, 则根据产生式的形式, 我们可以将文法划分为以下四种类型:

1. 类型 0 (Type 0): **无约束文法** (Unrestricted Grammars), 其产生式形式为 $\alpha \to \beta$, 其中 $\alpha \in (V \cup T)^+$, $\beta \in (V \cup T)^*$.

无约束文法可以描述所有可计算的语言, 但其解析复杂度较高, 我们通常不直接在实践中处理无约束文法.

2. 类型 1 (Type 1): **上下文相关文法** (Context-Sensitive Grammars, **CSG**), 其产生式形式满足 $|\beta| \geq |\alpha|$ 且 $\beta \neq \varepsilon$.

这里是“非收缩”定义. 另外有一种等价定义的产生式形式为 $\alpha_1 A \alpha_2 \to \alpha_1 \beta \alpha_2$, 其中 $A \in V$, $\alpha_1, \alpha_2 \in (V \cup T)^*$, $\beta \in (V \cup T)^+$. 这里 $|A| = 1$, $|\beta| > 1$, 且替换 A 时需要考虑其上下文 α_1 和 α_2.

3. 类型 2 (Type 2): **上下文无关文法** (Context-Free Grammars, **CFG**), 其产生式形式为 $A \to \beta$, 其中 $A \in V$, $\beta \in (V \cup T)^*$.

这里 A 是单个变量, 替换 A 时不考虑其上下文.

4. 类型 3 (Type 3): **正则文法** (Regular Grammars, **RG**), 其产生式形式为 $A \to aB$ 或 $A \to a$ 或 $A \to \varepsilon$, 其中 $A, B \in V$, $a \in T$.

正则文法是最简单的一类文法, 每次替换只能处理一个终结符, 并且可以选择性地跟随一个变量或结束替换过程.

9.6.2 线性文法

一个文法是 **线性文法** (Linear Grammar), 如果它的所有产生式都满足以下形式之一:

1. $A \to wB$
2. $A \to Bw$
3. $A \to w$

其中 $A, B \in V$, $w \in T^*$. 也就是说, 每个产生式的右部 **最多** 包含一个变量.

右线性文法 (Right-Linear Grammar) 是线性文法的一种特殊情况, 其中所有产生式都满足形式 $A \to wB$ 或 $A \to w$, 也就是说变量只能出现在最右侧; **左线性文法** (Left-Linear Grammar) 是线性文法的另一种特殊情况, 其中所有产生式都满足形式 $A \to Bw$ 或 $A \to w$, 即变量只能出现在最左侧.

一个文法是正则文法的充分必要条件是, 要么它的所有产生式都是右线性的, 要么它的所有产生式都是左线性的. **两种语法不能混合使用**, 否则可能生成非正则语言.

9.6.3 正则文法与正则语言的等价性

把右线性文法转换为 NFA:

1. 为文法的每个变量创建一个状态.
2. NFA 的初始状态对应文法的开始符号.
3. 对于每个产生式 $A \to aB$, 添加一个从状态 A 到状态 B 的转换, 输入符号为 a.
4. 对于每个产生式 $A \to a$, 添加一个从状态 A 到接受状态的转换, 输入符号为 a.

5. 如果存在产生式 $A \to \varepsilon$, 则将状态 A 标记为接受状态.

把 DFA 转换为右线性文法:
1. 为 DFA 的每个状态创建一个变量.
2. 文法的开始符号对应 DFA 的初始状态.
3. 对于每个转换 $\delta(q_i, a) = q_j$, 添加一个产生式 $A_i \to aA_j$, 其中 A_i 和 A_j 分别对应状态 q_i 和 q_j.
4. 对于每个接受状态 q_f, 添加一个产生式 $A_f \to \varepsilon$, 其中 A_f 对应状态 q_f.

左线性文法呢?

直接把左线性文法的所有产生式反转, 就能得到一个右线性文法. 因为正则语言对反转是封闭的, 所以左线性文法生成的语言也是正则语言.

9.7 空语句 (Null Sentence)

9.7.1 "不合法" 的规则

理论上来说, 产生空语句的规则 $A \to \varepsilon$ 是收缩的, 所以不属于类型 1 (上下文相关文法) 的定义范围. 但是在实际应用中, 我们通常允许文法包含产生空语句的规则, 因为空语句在很多语言中是有意义的.

9.7.2 准备工作: 如何处理开始符号

定理 1: 对于任何一个文法 G, 都存在一个等价的文法 G' 满足以下条件:
1. G' 中的开始符号 S' 不出现在任何产生式的右部.
2. $L(G') = L(G)$.

证明:
1. 创造一个全新的开始符号 S'.
2. 找到旧文法 G 中所有以旧开始符号 S 为左部的产生式, 并将它们复制到新文法 G' 中, 但将左部替换为 S'.
3. G' 包含旧文法 G 中的所有产生式, 以及我们新增的 $S' \to \cdots$ 规则.

这就相当于给一个公司重组: 原来的 CEO S 既要做决策 (出现在左部), 还要执行任务 (出现在右部). 为了让 CEO 专注于决策, 我们任命一个新的董事长 S', 其唯一工作就是任命 CEO S 来执行任务. 这样, 新的董事长 S' 就不会出现在任何任务描述中 (右部).

9.7.3 如何给一个语言添加空语句

如果一个语言 L 是某种类型 (比如说上下文无关语言 CFL), 则我们可以通过以下步骤构造一个新的文法 G' 来生成包含空语句的语言 $L' = L \cup \{\varepsilon\}$, 其类型保持不变:

1. 任何一个语言 L, 都有一个文法 G 生成它.
2. 根据定理 1, 构造一个等价的文法 G' 使得 G' 的开始符号 S' 不出现在任何产生式的右部.
3. 在 G' 中添加一个新的产生式 $S' \to \varepsilon$.
4. 因为 S' 不出现在任何产生式的右部, 所以添加 $S' \to \varepsilon$ 不会影响其他句子的生成, 它唯一的作用就是能直接产生一个空语句.
5. 所以新文法产生的语言 $L(G') = L(G) \cup \{\varepsilon\}$, 且 G' 的类型与 G 相同.

9.7.4 如何从一个语言中移除空语句

如果一个语言 L 包含空语句, 则我们可以通过以下步骤构造一个新的文法 G' 来生成不包含空语句的语言 $L' = L \setminus \{\varepsilon\}$, 其类型保持不变:

1. 如果 $\varepsilon \in L$, 则必须存在一个文法 G 生成 L 且包含一个产生式 $S \to \varepsilon$, 其中 S 是 G 的开始符号.
2. 利用定理 1, 构造一个等价的文法 G'' 使得 G'' 的开始符号 S' 不出现在任何产生式的右部.
3. 在这种情况下, $S' \to \varepsilon$ 这条规则是 "孤立" 的, 它只负责产生空语句. 所以我们可以直接删除这条规则.
4. 删除后的新文法产生的语言, $L(G') = L(G) \setminus \{\varepsilon\}$, 且 G' 的类型与 G 相同.

9.8 上下文无关文法 (CFG)

9.8.1 CFG 的派生树

在上下文无关文法 (CFG) 中, 我们可以使用 **派生树** (Derivation Trees) 来表示句子的生成过程. 派生树是一种树形结构, 其中:

- 每个内部节点表示一个变量 (非终结符).
- 每个叶子节点表示一个终结符 (终结符) 或空串 ε.
- 根节点表示开始符号.
- 从根节点到叶子节点的路径表示从开始符号到句子的推导过程.

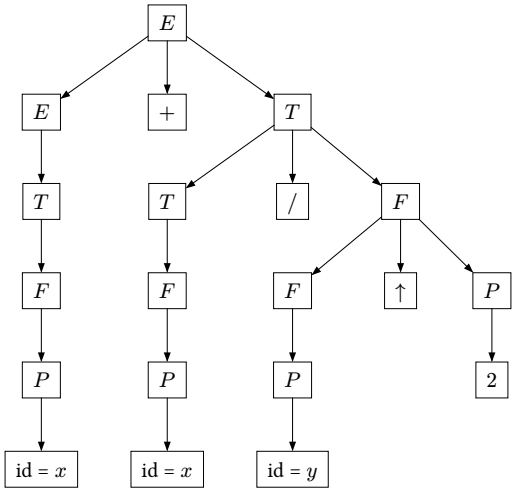
考虑下面算术表达式的文法

$$
\begin{aligned}
G: E &\to E + T \mid E - T \mid T \\
T &\to T * F \mid T/F \mid F \\
F &\to F \uparrow P \mid P \\
P &\to (E) \mid N(L) \mid \text{id} \\
N &\to \sin \mid \cos \mid \exp \mid \log \mid \ldots \\
L &\to L, E \mid E
\end{aligned}
\tag{26}
$$

其中

- E 表示表达式 (Expressions), 通过加法和减法组合.
- T 表示项 (Terms), 通过乘法和除法组合.
- F 表示因子 (Factors), 通过底数和指数组合.
- P 表示基本元素 (Primitives), 可以是括号内的表达式, 函数调用, 或标识符.
- N 表示函数名称 (Function Names).
- L 表示函数参数列表 (Argument Lists), 通过逗号分隔的表达式序列.

和表达式 $x + x/y \uparrow 2$:



注意派生树是有序的: 同一层级的节点从左到右的顺序与产生式中候选的顺序一致.

9.8.2 文法的二义性

对于计算机语言来说, 一条指令必须有且仅有一个确切的含义. 如果一个语法规则 (文法) 允许一个句子 (或一段代码) 被解释成两种或更多的含义, 那么这个文法就是 二**义性** 的.

既然二义性这么糟糕, 我们必须消除它. 方法有两种:

1. **外部规定**: 通过在文法之外添加额外的规则来消除二义性. 例如, 在算术表达式中规定乘法优先于加法, 这样就能消除表达式 $a + b * c$ 的二义性.

2. **修改文法本身**: 我们可以设计一套更精巧的语法规则, 让它本身就含有优先级的概念. 比如, 我们可以引入不同的变量来表示不同优先级的表达式, 例如:

$$
\begin{aligned}
E &\to E + T \mid T \\
T &\to T * F \mid F \\
F &\to (E) \mid \mathrm{id}
\end{aligned}
\tag{27}
$$

这样, 乘法 $T * F$ 总是在加法 $E + T$ 之内被解析, 从而消除了二义性.